

audio-hash-marker

Cài lab bằng imodule

Trên máy Labtainer, mở terminal ngoài container và chạy lệnh sau để tải bài lab từ GitHub RAW:

```
imodule https://github.com/vuxvinh/KiThuatGiauTin/raw/main/audio-hash-marker.tar
```

Sau khi imodule tải xong, chạy lab bằng lệnh:

```
labtainer audio-hash-marker
```

Nếu muốn chạy lại lab từ đầu, dùng lệnh sau:

```
labtainer -r audio-hash-marker
```

Khi terminal container student mở ra, chạy ls để kiểm tra các file bài lab.

Mục tiêu bài lab

Bài lab này hướng dẫn sinh viên giấu một thông điệp vào file WAV bằng kỹ thuật LSB, tự đánh dấu dữ liệu bằng marker AUDIO_SELF_MARK và kiểm tra toàn vẹn thông điệp bằng SHA-256.

- Task 1: Nhúng thông điệp vào input.wav để tạo output.wav.
- Task 2: Trích xuất marker, message và kiểm tra hash OK.
- Task 3: Chỉnh sửa nhẹ audio bằng tamper.py và phát hiện Hash check: FAILED.

Lưu ý: Lưu ý quan trọng: output.wav và tampered.wav không có sẵn lúc bắt đầu. Hai file này là kết quả sinh ra khi sinh viên chạy đúng các lệnh ở từng task.

Các file có sẵn trong container

```
README.md
make_assets.py
input.wav
embed.py
extract.py
tamper.py
check_work.sh
```

Sinh viên chỉ cần sửa hai file embed.py và extract.py. File tamper.py và check_work.sh đã được cung cấp sẵn để kiểm tra bài làm.

Bước 1: Kiểm tra file audio đầu vào

Trong container student, chạy lệnh sau để xem các file bài lab:

```
ls
```

Kiểm tra file WAV mẫu:

```
file input.wav
```

Kết quả cần cho thấy input.wav là file WAV audio. Đây là file âm thanh gốc dùng để nhúng tin.

Bước 2: Hoàn thiện embed.py

Mở file embed.py:

```
nano embed.py
```

Trong file này, sinh viên cần hoàn thiện hai hàm build_payload và embed_bits_into_audio.

2.1. Sửa hàm build_payload

Hàm này tạo dữ liệu cần giấu theo định dạng MARKER|MESSAGE|SHA256(MESSAGE). Thay phần TODO của hàm build_payload bằng đoạn code sau:

```
def build_payload(message: str) -> bytes:
    if DELIMITER in message:
        raise ValueError("Message must not contain the | delimiter")

    digest = hashlib.sha256(message.encode()).hexdigest()
    payload = f"{MARKER}{DELIMITER}{message}{DELIMITER}{digest}"
    return payload.encode()
```

Ý nghĩa: chương trình tính SHA-256 của message, sau đó ghép marker, message và hash thành một payload duy nhất để nhúng vào audio.

2.2. Sửa hàm embed_bits_into_audio

Hàm này nhúng từng bit của payload vào bit cuối cùng của từng byte audio. Thay phần TODO của hàm embed_bits_into_audio bằng đoạn code sau:

```
def embed_bits_into_audio(frame_bytes: bytes, payload: bytes) -> bytes:
    audio = bytearray(frame_bytes)
    bits = list(bytes_to_bits(payload))

    if len(bits) > len(audio):
        raise ValueError("Payload is too large for this audio file")

    for i, bit in enumerate(bits):
        audio[i] = (audio[i] & 0xFE) | bit

    return bytes(audio)
```

Công thức (audio[i] & 0xFE) | bit giữ nguyên 7 bit đầu của byte audio và chỉ thay đổi LSB theo bit cần giấu.

Bước 3: Chạy Task 1 để tạo output.wav

Sau khi sửa embed.py, chạy:

```
python3 embed.py input.wav output.wav "this is a secret"
```

Nếu đúng, chương trình sẽ in:

```
EMBED_OK
marker=AUDIO_SELF_MARK
message=this is a secret
output=output.wav
```

Lúc này file output.wav mới được tạo ra. Kiểm tra bằng lệnh:

```
ls -l output.wav
```

Bước 4: Hoàn thiện extract.py

Mở file extract.py:

```
nano extract.py
```

Trong file này, sinh viên cần hoàn thiện ba hàm `bits_to_bytes`, `extract_lsb_bytes` và `parse_payload`.

4.1. Sửa hàm `bits_to_bytes`

Thay phần TODO của hàm `bits_to_bytes` bằng đoạn code sau:

```
def bits_to_bytes(bits):
    data = bytearray()

    for i in range(0, len(bits), 8):
        chunk = bits[i:i + 8]

        if len(chunk) < 8:
            break

        value = 0
        for bit in chunk:
            value = (value << 1) | bit

        data.append(value)

    return bytes(data)
```

4.2. Sửa hàm `extract_lsb_bytes`

Thay phần TODO của hàm `extract_lsb_bytes` bằng đoạn code sau:

```
def extract_lsb_bytes(frame_bytes: bytes) -> bytes:
    bits = []

    for b in frame_bytes:
        bits.append(b & 1)

    return bits_to_bytes(bits)
```

4.3. Sửa hàm `parse_payload`

Thay phần TODO của hàm `parse_payload` bằng đoạn code sau:

```
def parse_payload(data: bytes):
    marker_prefix = f"{MARKER}{DELIMITER}".encode()
    marker_start = data.find(marker_prefix)

    if marker_start == -1:
        raise ValueError("Marker not found")

    message_start = marker_start + len(marker_prefix)
    delimiter_pos = data.find(DELIMITER.encode(), message_start)

    if delimiter_pos == -1:
        raise ValueError("Message delimiter not found")

    hash_start = delimiter_pos + 1
    hash_end = hash_start + HASH_HEX_LENGTH

    if hash_end > len(data):
```

```

        raise ValueError("Hash field not complete")

    message = data[message_start:delimiter_pos].decode()
    embedded_hash = data[hash_start:hash_end].decode()

    if len(embedded_hash) != HASH_HEX_LENGTH:
        raise ValueError("Invalid hash length")

    return MARKER, message, embedded_hash

```

Hàm này tìm marker AUDIO_SELF_MARK, lấy message nằm giữa hai dấu |, sau đó đọc 64 ký tự hash ở cuối packet.

Bước 5: Chạy Task 2 để trích xuất thông điệp

Chạy lệnh:

```
python3 extract.py output.wav
```

Kết quả đúng cần có:

```

Marker found: YES
Marker: AUDIO_SELF_MARK
Message: this is a secret
Hash check: OK

```

Nếu Hash check là OK, nghĩa là hash tính lại từ message trùng với hash đã được nhúng trong audio.

Bước 6: Chạy Task 3 để kiểm tra phát hiện chỉnh sửa

Chạy tamper.py để chỉnh sửa nhẹ hash đã nhúng trong audio:

```
python3 tamper.py output.wav tampered.wav
```

Sau lệnh này, file tampered.wav mới được tạo ra. Kiểm tra lại bằng extract.py:

```
python3 extract.py tampered.wav
```

Kết quả mong muốn:

```

Marker found: YES
Marker: AUDIO_SELF_MARK
Message: this is a secret
Hash check: FAILED

```

Điều này chứng minh marker và message vẫn còn đọc được, nhưng hash nhúng trong file đã bị thay đổi nên kiểm tra toàn vẹn thất bại.

Bước 7: Chạy check_work.sh

Sau khi hoàn thiện embed.py và extract.py, chạy:

```
bash check_work.sh
```

Nếu đúng toàn bộ, kết quả cần có:

```

TASK1_EMBED_PASS
TASK2_EXTRACT_PASS
TASK3_TAMPER_DETECTED_PASS
ALL_TASKS_PASS

```

Nếu task nào chưa đạt, check_work.sh sẽ in FAIL ở đúng task đó để sinh viên biết cần sửa phần nào.

Bước 8: Chấm bằng checkwork

Từ terminal Labtainer, chạy:

```
checkwork audio-hash-marker
```

Kết quả mong muốn:

Task_1_Embed_secret_message	Y
Task_2_Extract_and_verify_hash	Y
Task_3_Detect_tampered_audio	Y
All_tasks_completed	Y

Câu hỏi báo cáo

1. Kỹ thuật LSB trong audio hoạt động như thế nào?
2. Vì sao thay đổi LSB thường khó nhận biết bằng tai người?
3. Marker AUDIO_SELF_MARK dùng để làm gì?
4. Vì sao cần gắn SHA-256 vào message?
5. Hash check: OK có nghĩa là gì?
6. Hash check: FAILED có nghĩa là gì?
7. Vì sao sau khi tamper, marker và message vẫn có thể đọc được nhưng hash check lại FAILED?
8. Kỹ thuật này có đảm bảo bí mật tuyệt đối không? Vì sao?
9. Nếu message chứa ký tự | thì chương trình có thể gặp lỗi gì?
10. Làm thế nào để cải tiến định dạng packet để hỗ trợ mọi loại message?